

Reliability of Microscaling-Format Neural Networks under Bit Faults

Peeyush*

22 September 2026

1 Background and theory

Low-precision inference is limited by dynamic range: a 4- or 6-bit number cannot span the spread of values in a weight tensor. Microscaling (MX) formats solve this by sharing one scale across a small block. A block of K elements is stored as one 8-bit exponent X (the E8M0 scale) together with K narrow element codes m_i , and the hardware reconstructs

$$v_i = m_i \cdot 2^{X-127}, \quad i = 1, \dots, K. \quad (1)$$

An element code is a small sign-exponent-mantissa number decoding to $(-1)^s(1+f/2^M)2^{e-\text{bias}}$ for $e > 0$; e4m3 uses 1+4+3 bits, e3m2 1+3+2, e2m1 1+2+1. The two 8-bit formats reserve codes for NaN and infinity, whereas the 6- and 4-bit formats have none, so every one of their bit patterns is a finite number. That distinction turns out to matter more than any other. The scale follows the OCP rule $X = \lfloor \log_2 \max_i |v_i| \rfloor - e_{\max}$, with $e_{\max}$ the exponent of the largest value the element format can hold.

In ordinary floating point a value is a self-contained word, so a bit flip perturbs one number. Equation (1) breaks that. An *element* fault flips a bit of one m_i : a mantissa bit changes the value by at most a relative 2^{j-M} , an exponent bit multiplies it by $2^{\pm 2^j}$, and the sign bit negates it. A *scale* fault flips a bit of X , so $X \rightarrow X \pm 2^j$ and all K elements are multiplied by $2^{\pm 2^j}$ at once; if the flip produces the reserved code 255 the whole block becomes NaN. The number of values a fault alters is its *blast radius*: 1 for an element fault, K for a scale fault.

This predicts something before any measurement. A block occupies $8 + Kw$ bits (w = element width), of which only 8 belong to the scale, about 3% for $K = 32, w = 8$. Yet if every stored bit is equally likely to flip, the expected number of corrupted *values* per block is $8K$ from scale faults against Kw from element faults, a ratio of $8 : w$. An 8-bit format therefore takes about half its corrupted values through a scale occupying 3% of storage, and a 4-bit format two thirds.

2 Fault model and metrics

I use the single-bit-flip model standard in fault-injection work, applied to stored values: one bit of one element code, or of one scale byte, is inverted. Weight faults are persistent and affect every inference; activation faults are transient and affect only the inference in which they occur. Let $D = \{(x_i, y_i)\}_{i=1}^N$ be a fixed evaluation set, $\hat{y}_i$ the prediction of the fault-free but MX-quantised network, and $\hat{y}_i^\varphi$ the prediction under fault φ .

metric	definition	meaning
corruption count	$c(\varphi) = \sum_i \mathbb{K}[\hat{y}_i^\varphi \neq \hat{y}_i]$	inferences the fault changed
SDC indicator	$s(\varphi) = \mathbb{K}[c(\varphi) \geq 1]$	did it corrupt anything at all
SDC rate	$\hat{p} = \Phi ^{-1} \sum_{\varphi \in \Phi} s(\varphi)$	fraction of faults that corrupt
per-inference rate	$r(\varphi) = c(\varphi)/N$	chance a given inference is corrupted
non-finite rate	$ \Phi ^{-1} \sum_{\varphi} \mathbb{K}[\text{NaN/Inf output}]$	faults that destroy the output

The SDC rate counts a fault as a failure if it changes even one prediction, so it is deliberately sensitive and comparable across configurations rather than across papers. Weight and activation faults must not be compared on it, since a weight fault is exposed to all N inferences and an activation fault to one; the comparable quantity is $r(\varphi)$. Rates carry Wilson 95% intervals. Because the element alphabets hold at most 256 codes, the decoded value of every (code, bit) pair is tabulated exactly rather than sampled.

3 Method

The campaign compares a golden run against a faulty one. The network is first put in the form an accelerator executes: batch normalisation folded into the preceding convolution, and RepVGG's branches fused. This matters because an unfolded batch norm re-normalises whatever the corrupted convolution produced and hides

*Replace with your full name and roll number.

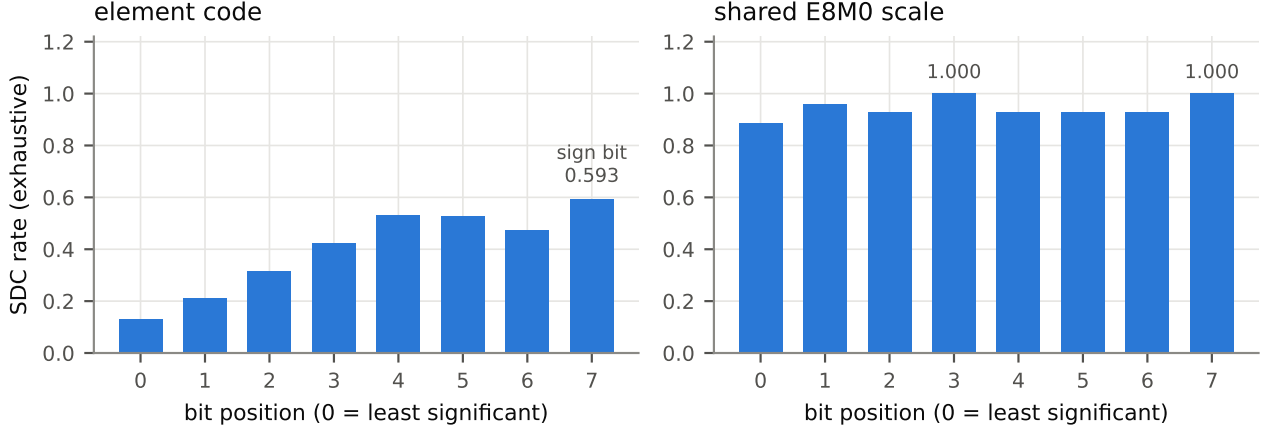


Figure 1: Exhaustive per-bit failure rates (ResNet8, e4m3, $K = 32$, 638,624 injections). Damage grows with bit significance inside an element code, but the sign bit is the worst position of all. Every bit of the shared scale exceeds 0.87, and bit 3 (a factor 256) and bit 7 (overflow past the float32 range) never leave a prediction intact.

part of the damage. Weights are then encoded into real MX bit patterns, the fault is applied *in the code domain*, and the block is decoded only afterwards, which is what lets a scale fault perturb all K values.

Algorithm 1 Fault-injection campaign

Require: network M , format F , block size K , evaluation set D , budget n

- 1: $M \leftarrow \text{DEPLOY}(M)$ ▷ fold batch norm, fuse branches
- 2: **for all** quantisable layers ℓ **do**
- 3: view W_ℓ as (outputs $\times$ reduction), split the reduction axis into blocks of K
- 4: $X_\ell \leftarrow \lfloor \log_2 \max |W_\ell| \rfloor - e_{\max}$ per block; $C_\ell \leftarrow \text{round}_F(W_\ell 2^{-X_\ell})$
- 5: $W_\ell \leftarrow \text{DECODE}(X_\ell, C_\ell)$ ▷ the network now holds MX values
- 6: **end for**
- 7: $\hat{y} \leftarrow \arg \max M(D)$ ▷ golden predictions
- 8: **for** $t = 1$ to n **do**
- 9: draw $\varphi = (\ell, \text{site}, \text{index}, b)$ uniformly over all stored bits
- 10: **if** site = element **then** $C_\ell[\text{index}] \leftarrow C_\ell[\text{index}] \oplus 2^b$; update that one weight
- 11: **else** $X_\ell[\text{index}] \leftarrow X_\ell[\text{index}] \oplus 2^b$; update the K weights of the block
- 12: **end if**
- 13: $\hat{y}^\varphi \leftarrow \arg \max M(D)$; record $c(\varphi)$, $s(\varphi)$, non-finiteness; restore weights
- 14: **end for**
- 15: **return** rates over Φ with Wilson intervals

Line 8 defines the sampling. Drawing bits uniformly is the physically faithful choice, since faults land in proportion to storage; replacing that line by an enumeration of every addressable bit makes the campaign *exhaustive*, giving the true failure rate with no sampling error.

Setup. CIFAR-10 with ResNet8 (78k parameters, 87.0% top-1), RepVGG-A0 (7.0M, 91.5%, three seeds) and a vision transformer (2.7M, DeiT-Tiny width, 81.9%, three seeds), plus ResNet8 at five widths from 20k to 692k with ten independently trained models each. Formats e4m3, e5m2, e3m2, e2m3, e2m1, block sizes 8–64. The evaluation set is a fixed class-balanced subset of 200 test images; campaigns use 3000 injections unless stated otherwise.

4 Results

Exhaustive ground truth. Flipping all 638,624 bits of ResNet8’s MXFP8 weight storage gives a true SDC rate of 0.4171. The ordinary 3000-injection campaign returned 0.4270 with interval $[0.4094, 0.4448]$: 2.4% off in relative terms, interval containing the truth. Repeating the enumeration under e3m2 gives a second exact reference on the same network (483,904 injections, rate 0.1079) and a second confirmation of the sampling (estimate 0.1070, interval $[0.0964, 0.1186]$). Exactly, under e4m3 element faults corrupt at 0.4003 and scale faults at 0.9442, with 3.27 against 49.89 of 200 inferences corrupted. Vulnerability runs opposite to layer size: the 3584-bit stem is the most fragile layer at 0.853, the 304,128-bit final convolution the most robust at 0.289, so a uniform campaign spends 48% of its budget on the most robust layer.

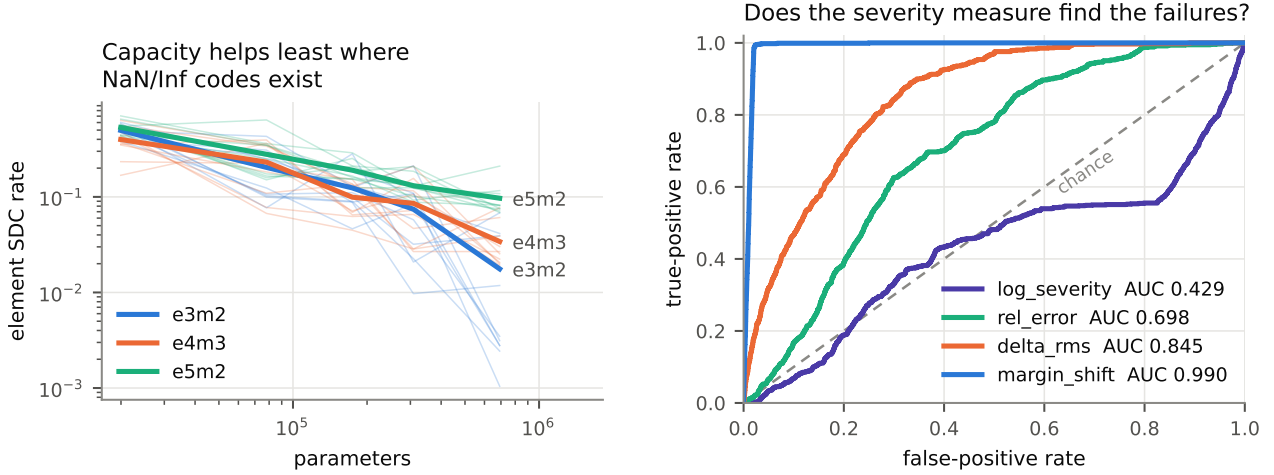


Figure 2: Left: element SDC against parameter count, ten independently trained models per width drawn faintly behind each mean. Capacity helps least for the format with the most NaN/Inf codes. Right: how well each severity measure ranks the faults that actually failed. The measure proposed in the project plan falls below the chance diagonal; margin_shift reaches AUC 0.990.

Accuracy does not predict reliability, for two separate reasons. On the transformer every format except MXFP4 lands within 0.2 points of fp32 accuracy, so e5m2 and e3m2 differ by 0.01 points – and their element SDC rates are 0.0730 ± 0.0089 and 0.0041 ± 0.0047 , a factor of 17.8. Two independent mechanisms produce such gaps.

Special codes. Only formats owning NaN/Inf codes can produce a non-finite value, and nothing downstream absorbs one. On the transformer 94% of e5m2’s element failures were non-finite events, and on RepVGG 92%, against none for e3m2 in any campaign or in the 483,904-fault exhaustive enumeration. Training ResNet8 at five widths, ten seeds each, isolates this: the non-finite rate is a property of the format, varying under 20% across a 35-fold change in capacity with no correlation against model size, while the benefit a format draws from capacity falls with its number of special codes (Figure 2, left).

Sensitivity, and what it is not. A second mechanism appears among formats with no special codes at all. Perturbing a single weight by a fixed multiple of the layer RMS – a probe containing no bit patterns – separates the quantised networks completely: at 1.5 RMS the e3m2 and e5m2 networks never flipped a prediction in 1500 trials, while e4m3 flipped 2–5% of the time. The fragility belongs to the network, not the encoding, and its cause is the sensitivity $s_i = |\partial m / \partial w_i| \text{rms} / m$, with m the margin between the top two logits: a perturbation of $\delta \cdot \text{rms}$ flips a prediction when $s_i \delta > 1$, and computing s by backpropagation reproduces the probe with correlation +0.998. Across the five formats s predicts the failure rate at $r = +0.925$. Two things it is *not*. The tail of s is not inherited from the tails of the weights, activations or output gradients, which agree across formats to within 3% while s differs 14-fold at the 99th percentile. And it is not mantissa width, which correlates at -0.251 once MXFP4 is included – e2m1 has the fewest mantissa bits and the highest failure rate. Within the four 6- and 8-bit formats the 2-mantissa pair is genuinely 6–8 $\times$ safer, confirmed exactly by enumerating one ResNet8’s entire fault space under both (0.0785 against 0.4003), but that is a correlate of the group rather than a rule.

Shared scales dominate on every architecture. Scale faults outweigh element faults by 2.4 $\times$ on ResNet8 (exhaustive, e4m3), 28.7 $\times$ on RepVGG-A0 and 12.1 $\times$ on the transformer, and they corrupt 16–50 of 200 inferences each. Where the element rate is very low the ratio becomes extreme – 232 $\times$ for e3m2 on RepVGG – because the scale is then almost the only way to break the network. Scale faults also produce non-finite outputs at 7–13% *for every format*, including those with no special element codes, because the E8M0 scale has its own NaN code and an MSB worth 128 exponent steps.

Weights matter far more than activations. On the comparable per-inference rate, element faults are 5.3 $\times$ more damaging in weights than activations on ResNet8 and 9.3 $\times$ on RepVGG; on the transformer *none* of 2000 activation faults changed its own inference (interval $[0, 0.0020]$). A transient fault in one activation value is a minor concern next to a persistent weight fault, which every inference reads and which, at the scale site, corrupts K values at once.

Block size, and how much one model can be trusted. Raising K from 8 to 64 cuts the storage share of scale bits from 11.1% to 1.7% while a scale fault’s SDC rate rises from 0.731 to 0.941, with accuracy nearly flat in K : a clean efficiency-against-reliability trade. Across ten seeds per width, the spread between independently

trained models is about ten times the uncertainty contributed by 3000 injections, so a reliability figure quoted from one trained model is far less certain than its injection count suggests.

5 Two problems with the proposed methodology

The severity measure, and a replacement. The plan scores an element fault by $d = |\log_2 |v'| - \log_2 |v||$. Over 39,337 element faults its rank correlation with observed SDC is -0.035 and its AUC is 0.435 – it ranks failures slightly *worse* than chance (Figure 2, right). The reason is in the definition: negating a value leaves $|v|$ unchanged, so a sign flip scores zero severity although the exhaustive data show the sign bit to be the most damaging element bit. Three successively better measures follow: relative error (0.699), absolute change in layer-RMS units (0.845), and $\text{margin_shift} = s_i |v' - v| / \text{rms}$, the fraction of the decision margin a fault consumes, at AUC 0.990 . Its threshold needs no tuning: flagging faults with $\text{margin_shift} > 1$ catches 98.5% of all failures while flagging 4.4% of the fault space, which is directly usable for the value-aware sampling the plan wanted. It also carries a caveat the original formulation hides: severity is not a property of the number system, since the same flip in the same format behaves very differently in two networks of equal accuracy.

The standard scale rule manufactures a trend. Because the largest representable element lies below the top of the binade the OCP rule aligns to, the block maximum is clipped, and smaller blocks mean more maxima and more clipping. Injecting an identical set of faults at every block size, element-side SDC varies by a factor 2.21 on ResNet8 and 7.25 on RepVGG-A0 under the OCP rule, but is flat (1.04 and 1.48) under a rule that cannot clip. The apparent best block size for reliability is an artefact of the scale rule, so every result obtained under it needs a non-clipping control.

6 Status

The study covers three architecture families and answers all five objectives. Design guidance follows directly: prefer element formats without NaN/Inf codes, protect the shared scale before anything else, harden weights before activations, and measure the sensitivity s of the quantised network rather than inferring robustness from a format’s field widths – two formats of identical width and accuracy differ by an order of magnitude. The gap is scale: everything is CIFAR-10, because ImageNet is not available on the hardware I can reach. One question stays open – why the sensitivity tail differs at all between quantised networks whose weight, activation and gradient distributions are indistinguishable.